

TENPROF: A Tensor-Centric Profiler for Deep Learning Workload Analysis and Optimization

Xingjian Ding

University of California, Merced
Merced, CA, USA
xding8@ucmerced.edu

Yueming Hao

Meta
Menlo Park, CA, USA
yhao@meta.com

Keren Zhou

George Mason University
Fairfax, VA, USA
kzhou6@gmu.edu

Pengfei Su

University of California, Merced
Merced, CA, USA
psu9@ucmerced.edu

Abstract

Deep learning (DL) workloads span multiple layers of abstraction, from high-level Python scripts to intermediate C/C++ tensor libraries and low-level GPU compute kernels. While this layered design enables flexibility, it also obscures the root causes of inefficiencies, leaving GPUs underutilized. Profilers are essential for exposing such inefficiencies, yet existing tools fall short of explaining how high-level DL semantics translate into low-level GPU kernel and hardware behavior. They either offer DL operator-level statistics (e.g., for a convolution operator) with coarse-grained metrics such as execution time and memory usage, or focus exclusively on individual GPU kernels without linking them to DL semantics. As a result, true performance bottlenecks remain hard to diagnose and fix.

We present TENPROF, the first tensor-centric GPU profiler for PyTorch-based DL workloads. TENPROF systematically bridges low-level hardware stall metrics with high-level DL semantics (particularly tensor transformations) through three key tensor-centric analyses: (i) tracing tensor transformations and modeling them as a forest, (ii) attributing GPU memory accesses to both base and view tensors, and (iii) assessing tensor access efficiency via hardware stall attribution. By unifying insights across layers, TENPROF reveals how tensor accesses and transformations directly impact performance. It operates on unmodified program binaries, introduces moderate overhead, and generates interpretable profiles that guide actionable optimizations.

Evaluation across several real-world DL applications shows that TENPROF uncovers inefficiencies such as non-contiguous memory accesses, where resolving them requires only minor code changes yet yields non-trivial speedups. Several optimizations guided by TENPROF have already been validated and upstreamed by application developers, demonstrating its practical value in production settings.

CCS Concepts

• **General and reference** → **Performance**; **Measurement**; • **Computer systems organization** → *Parallel architectures*; • **Computing methodologies** → *Parallel programming languages*.

Keywords

Profiling tools, Deep learning, Performance tuning, Tensor-centric analysis, GPUs, PyTorch

ACM Reference Format:

Xingjian Ding, Keren Zhou, Yueming Hao, and Pengfei Su. 2026. TENPROF: A Tensor-Centric Profiler for Deep Learning Workload Analysis and Optimization. In *2026 International Conference on Supercomputing (ICS '26)*, July 06–09, 2026, Belfast, United Kingdom. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3797905.3800515>

1 Introduction

Deep learning (DL) has emerged as the dominant paradigm for building intelligent systems, with applications ranging from autonomous vehicles [50] and medical imaging [18, 19, 67] to large language models [51] and scientific computing [26]. Unlike traditional workloads, DL workloads operate across multiple layers of abstraction and programming paradigms. These layers include the high-level Python layer (e.g., model definitions and training scripts), the intermediate C/C++ layer (e.g., PyTorch tensor libraries [4, 54]), and the low-level GPU kernel layer (e.g., NVIDIA cuDNN [6] and AMD MIOpen [28]). This multilayered design is necessary to cater to the diverse needs of different application domains, but it also introduces performance inefficiencies that are deeply embedded within and across layers. These inefficiencies are often difficult to pinpoint and prevent GPUs from achieving their full potential.

Despite rapid advances in DL compiler ecosystems, such as TorchDynamo [57], TorchInductor [25], Triton [65], TVM [5], OpenXLA [53], and IREE [23], optimizing the computations within complex workloads remains elusive. Prior studies have shown that Model FLOPs Utilization on GPUs often reaches only about 30% of peak for large models [7, 55], highlighting significant untapped performance. This gap underscores the need to systematically identify and eliminate inefficiencies in DL workloads to approach bare-metal execution. Performance profiling tools (aka profilers) are indispensable in this process: they complement compilers by surfacing hidden inefficiencies, attributing costs to the responsible



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICS '26, Belfast, United Kingdom*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2522-7/2026/07

<https://doi.org/10.1145/3797905.3800515>

layers or operations, and providing actionable insights that guide both manual tuning and compiler-driven optimizations.

Inadequacies of existing profilers. State-of-the-art DL profilers, such as PyTorch Profiler [35], JAX Profiler [15], and TensorBoard [14], primarily surface coarse-grained metrics (e.g., operator execution time) without exposing deeper microarchitectural insights like cache misses, instruction stalls, uncoalesced memory accesses, and warp divergence. In practice, execution time alone is a blunt diagnostic tool: a long-running operator may simply reflect large, already well-optimized computation, while a short-lived operator (e.g., a small tensor transpose) can silently introduce non-contiguous memory accesses that severely cripple the efficiency of downstream operators. Because these microarchitectural insights are not surfaced, users often misdiagnose bottlenecks or waste time optimizing the wrong part of the workload.

Moreover, existing DL profilers adopt a code-centric view, attributing performance to DL operators or GPU kernels. This perspective fails to align with the tensor-centric nature of modern DL workloads, where tensor shapes, layouts, transformations, and access patterns fundamentally influence efficiency. For example, a seemingly innocuous tensor transformation (e.g., `permute()` in PyTorch) may generate a tensor with the non-contiguous layout that degrades memory coalescing and throughput. Yet such effects remain invisible in the metrics exposed by existing DL profilers, leaving practitioners with little visibility into how such tensor transformations propagate and amplify inefficiencies.

In contrast, general-purpose GPU profilers, such as NVIDIA’s Nsight Compute [45], leverage hardware features like program counter (PC) sampling [43] to collect fine-grained microarchitectural insights, including instruction stalls with reasons (e.g., synchronization and memory dependency) recorded. However, these tools are designed around classical GPU programming models (e.g., CUDA) and lack semantic alignment with high-level DL abstractions. Consequently, interpreting instruction stalls in terms of tensor inefficiencies remains challenging. For instance, detecting that a matrix multiplication kernel is bottlenecked by memory dependency is informative; however, realizing that the root cause lies in a particular upstream tensor reshape or broadcasting pattern yields more actionable guidance.

In summary, existing profilers lack an abstraction that systematically connects low-level hardware inefficiencies with high-level DL semantics.

1.1 Solution Overview

To address the gaps in existing profilers, we present TENPROF¹, the first tensor-centric GPU profiler for PyTorch-based DL workloads. TENPROF introduces a new profiling paradigm that elevates tensors to first-class performance objects through three complementary analyses:

- **Analysis I: Tracing tensor transformations.** Tensors carry both data and metadata. The metadata describes a tensor’s size, stride, data type, start address, and other properties. Modern DL workloads rely heavily on lightweight view operations, such as slicing, permuting, splitting, and transposing, to perform fast tensor

transformations. These operations create *view tensors* that share the underlying storage of a base tensor² by modifying only the metadata, without moving data. While efficient, such metadata-only transformations can silently introduce non-contiguous, unaligned, or stride-increasing memory accesses that degrade performance. To capture these effects, TENPROF constructs a *transformation tree* for each base tensor, tracing how metadata evolves throughout execution. Collectively, these trees form a *transformation forest*.

- **Analysis II: Capturing tensor accesses.** To link memory accesses to specific tensors, TENPROF intercepts tensor allocations to record their storage address ranges and instruments GPU memory instructions (i.e., loads and stores) to capture the actual address ranges they access. By intersecting these two sets of address ranges, TENPROF precisely identifies not only which tensors are accessed, but also which subregions within them. This enables a fine-grained characterization of tensor access patterns such as strided and scattered accesses, which existing DL profilers cannot observe.
- **Analysis III: Assessing tensor access efficiency.** Using PC sampling and backward slicing, TENPROF identifies memory instructions that incur hardware stalls. By mapping these sampled instructions to the tensors they access (as determined in **Analysis II**), TENPROF distinguishes inefficient tensor accesses from benign ones. These inefficiencies are then linked to the responsible transformations using the tensor transformation tree constructed in **Analysis I**.

Together, these three analyses form an integrated tensor-centric profiling pipeline. They provide a complete view of performance: from low-level hardware metrics (e.g., memory-related stalls), to the specific tensor accesses that trigger them, to the high-level DL semantics (e.g., tensor transformations) that introduce these inefficiencies. This end-to-end linkage enables TENPROF to generate interpretable profiles that reveal not only *what* is slow, but *why*, a capability missing from existing DL profilers. For example, if TENPROF observes stalls on strided accesses to a tensor, users can trace back through its transformation tree to determine whether a stride-changing transformation (e.g., `transpose()`) introduced the inefficiencies.

In the remainder of this section, we first describe a motivating example, showing the unique performance insight enabled by the combination of TENPROF’s three analyses. We then summarize the contributions of this paper.

1.2 Motivating Example

Sana [48], an efficient high-resolution image synthesis built on linear diffusion transformers, is powered by NVIDIA. Listing 1 presents a representative code snippet from the `SanaLinearAttnProcessor2_0` module, where performance degradation is observed.

Using TENPROF, we find that the accesses to the `hidden_states` tensor on the right-hand side of line 7 are non-contiguous (identified by **Analysis II**), which incur substantial hardware stalls

¹ TENPROF is open-sourced at <https://github.com/David-Dingle/TenProf>.

²Throughout this paper, “tensor” refers to either a base or a view tensor unless explicitly prefixed with “base” or “view”.

(identified by **Analysis III**). By inspecting `hidden_states`'s transformation history (recorded in **Analysis I**), we observe that `hidden_states` is derived via the view operators `flatten()` and `transpose()` at line 5, followed by a `to()` operator for type casting at line 6. Although `to()` materializes the tensor, it preserves the non-contiguous layout caused by `transpose()`. This transformation trace reveals that `transpose()` is the root cause of the stalls. Because GPUs are optimized for coalesced memory accesses: threads within a warp accessing adjacent addresses. A transposed tensor destroys this contiguity, causing uncoalesced accesses and increased memory latency, which manifests as hardware stalls.

A simple fix is to call `contiguous()` immediately after `transpose()`. This call materializes the tensor into a new contiguous buffer, restoring coalesced access and yielding a 1.06× whole-module speedup.

```
1 class SanaLinearAttnProcessor2_0:
2     def __call__(self, attn: Attention, ...) -> torch.Tensor:
3         ...
4         hidden_states = hidden_states[:, :, :-1] / (
5             hidden_states[:, :, -1:] + 1e-15)
6         hidden_states = hidden_states.flatten(1, 2).
7             transpose(1, 2)
8         hidden_states = hidden_states.to(original_dtype)
9         hidden_states = attn.to_out[0](hidden_states)
```

Listing 1: Accesses to the transposed `hidden_states` tensor cause substantial hardware stalls in Sana. A transposed tensor has non-contiguous access patterns, which breaks coalescing and thus increases memory latency.

1.3 Contribution Summary

In this paper, we make the following contributions:

- We identify limitations in existing profilers, showing that current approaches lack a unifying abstraction that relates low-level hardware behavior to high-level DL semantics.
- We introduce the concept of *tensor-centric profiling*, embodied in TENPROF, which stitches three traditionally disjoint views of DL execution — tensor transformations, tensor-level memory accesses, and hardware performance behavior. This new perspective enables principled reasoning about how tensor transformations and access patterns directly impact performance.
- We show that TENPROF operates transparently on unmodified binaries, unifying different layers of DL workloads on the same level of abstraction. It requires no recompilation, introduces moderate overhead, and produces intuitive insights that guide performance optimization, making it applicable in the production environment.
- We demonstrate that TENPROF uncovers previously invisible inefficiencies in real-world DL applications. Resolving these issues typically requires only a few lines of code changes, yet delivers meaningful speedups. Several TENPROF-guided optimization patches have been validated by application developers and upstreamed into their repositories, underscoring the practical value of TENPROF.

2 Background

This section introduces off-the-shelf facilities that we leverage to design TENPROF, along with background knowledge necessary to understand our work.

2.1 NVIDIA Facilities

PC Sampling API. NVIDIA GPUs offer PC Sampling API [43], part of the CUDA Profiling Tools Interface (CUPTI) [40], to enable fine-grained, instruction-level profiling of GPU execution. It periodically samples PC values during kernel execution, along with associated stall reasons (e.g., memory dependency, execution dependency, and synchronization) and execution context. This enables statistical attribution of performance bottlenecks to individual instructions and links them to source code, making it useful for analyzing low-level GPU pipeline behavior.

Compute Sanitizer API. Compute Sanitizer API [39] is a dynamic binary instrumentation framework that enables users to build custom instrumentation tools in C/C++ and CUDA. It provides three classes of APIs: callback, patching, and memory APIs. The callback API allows users to register callbacks that are associated with specific CUDA runtime or driver functions (e.g., kernel launches, `cudaMalloc()`, and `cudaMemcpy()`). These callbacks are invoked when the associated CUDA functions are executed. The patching API enables users to inject custom code patches at selected instrumentation points, such as GPU memory instructions. When an instrumented instruction executes, the patch captures the accessed memory addresses and exposes them to users, enabling fine-grained observation of dynamic execution behavior. Finally, the memory API provides replacements for CUDA memory-management functions that can be safely invoked from within instrumentation tools. For example, `sanitizerAlloc()` serves as a safe alternative to `cudaMalloc()`, allowing instrumentation code to allocate memory without interfering with the application's execution.

Checkpoint API. Checkpoint API [42], shipped with CUPTI, enables controlled replay of GPU kernels by allowing users to save and restore execution checkpoints. A checkpoint is a copy of user-visible device state at a specific moment, such as values in memory and registers. Restoring this state allows kernels to be re-executed from an identical starting point, eliminating variability introduced by prior kernel executions. This capability is particularly useful for reproducible testing and performance measurements.

2.2 PyTorch Internals

Core code structure. PyTorch's codebase [60] is organized into a layered structure that mirrors its runtime architecture, including the high-level Python layer, the intermediate C++ layer, and the low-level device-specific layer. The Python layer (location: `torch/`) exposes the user-facing API for tensor creation, neural network modules (`torch.nn`), optimizers, and automatic differentiation. This layer is responsible for defining model structure and orchestrating execution, but delegates performance-critical work to lower layers.

The C++ layer, including `Aten` (location: `aten/`) and `C10` (location: `c10/`), implements tensor operations, metadata handling, kernel dispatch, and backend integration. `Aten` (short for a tensor library) defines a comprehensive set of tensor operators, enabling

mathematical and DL operations to be executed in both eager and graph-compiled modes. Each ATen operator is dispatched to one or more device-specific kernels. C10, named as a pun on Caffe2 and A“Ten” [11], provides the foundational abstractions for PyTorch, including tensor implementation, type and device management, and memory allocation.

Finally, the device-specific layer (locations: aten/src/ATen/native/) implements device-specific kernels for Aten operators, optimized for the underlying hardware’s execution model. These kernels realize the actual computation and memory accesses observed at runtime.

Tensors. A tensor, which comprises both data and metadata, is implemented as a lightweight handle in Python (`torch.Tensor`) backed by a C++ `at::Tensor` object. The `at::Tensor` holds an intrusive reference (intrusive_ptr) to a `TensorImpl` object, which encapsulates all metadata required to interpret the tensor’s data. The `TensorImpl` itself maintains an intrusive reference to a `StorageImpl` object, which manages the underlying raw memory buffer on the device.

Metadata. Metadata is encapsulated in the `TensorImpl` object, which describes a tensor’s start address, storage offset, data type, dimensionality, sizes, strides, etc. The *start address* specifies where the tensor’s data begins. The *storage offset* indicates the starting point of the tensor’s data within the underlying storage, measured as the number of elements from the absolute beginning of the storage; for a non-sliced tensor, this offset is 0. The *data type* defines the element representation (e.g., `float32` or `float16`). The *dimensionality* denotes the number of tensor axes, while the *sizes* specify the number of elements along each dimension. The *strides* describe the number of elements to skip along each dimension to move to the next element in that dimension.

View transformations. PyTorch programmers can apply fast view operators such as slicing, transposing, permuting, or other element-wise transformations to generate a series of *view tensors* by merely altering the tensor’s metadata, without copying data. As shown in Figure 1, the root represents a base tensor, which directly owns its underlying storage buffer (`StorageImpl`). Each child node corresponds to a view tensor derived from the base tensor. These view tensors share the same underlying storage buffer (`StorageImpl`) with the base tensor but maintain distinct metadata (`TensorImpl`) that define different logical shapes and layouts. Edges represent view operators, i.e., the transformations applied between tensors. A full list of view operators is available on the PyTorch website [61].

This design enables efficient tensor manipulation with minimal overhead, but it also allows complex chains of transformations to accumulate. Consequently, subtle changes in metadata such as stride reordering can significantly affect memory access behavior and downstream performance. *This observation motivates TENPROF’s approach of explicitly tracing tensor transformation histories to enable precise attribution of performance inefficiencies.*

Profiling hooks. PyTorch provides the `RecordFunctionCallback` API [62], which allows profilers to register custom callback functions that are invoked upon entry to and exit from profiled regions. These regions correspond to Aten

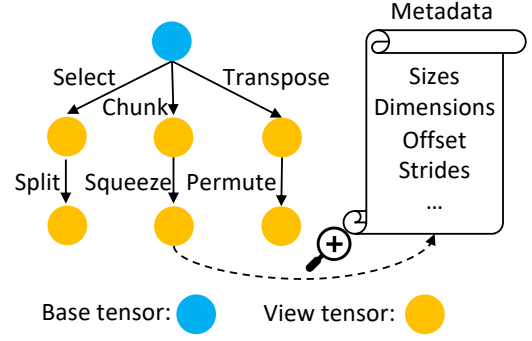


Figure 1: View transformations. The root represents a base tensor. Each child node corresponds to a view tensor derived from the base tensor. Edges represent view operators (transformations).

operators (i.e., tensor operators) or user-defined scopes. Using this API, one can hook into tensor operators’ call and return events, capturing their input and output tensors and thereby enabling precise tracking of tensor transformations.

3 TENPROF Workflow

TENPROF is designed to be practical for production environments. It operates transparently on unmodified binaries and requires no changes to hardware or operating systems. Figure 2 presents an overview of TENPROF’s workflow, which consists of an online analyzer and an offline GUI.

Online analyzer. The analyzer leverages PyTorch `RecordFunctionCallback` API [62] to hook into Aten operators’ call and return events to trace tensor transformations. It further leverages `Compute Sanitizer` API [39] and `CUPTI PC Sampling` API [43] to capture memory instructions that incur hardware stalls and to associate these instructions with the tensors being accessed. Together, these mechanisms pinpoint tensor transformations that cause inefficient memory accesses. In addition, TENPROF constructs cross-layer call paths for tensor allocations and accesses, providing a holistic view of execution across the DL software stack. This includes the upstream Python call path, the intermediate C++ call path, and the downstream CUDA call path. The Python call path is collected using the `PyEval_GetFrame` API [56], the C++ call path via the `libunwind` API [13], and the CUDA call path through the `CUPTI Activity` API [41].

Offline GUI. The GUI is built atop Graphviz [12], which adopts the DOT language as its input format and visualizes structured information as diagrams of abstract graphs and networks. Figure 7 in Section 6.1 shows an example of the GUI, which visualizes tensor transformations and accesses along with their call paths and stall percentages. We defer the explanation of the GUI details to that section.

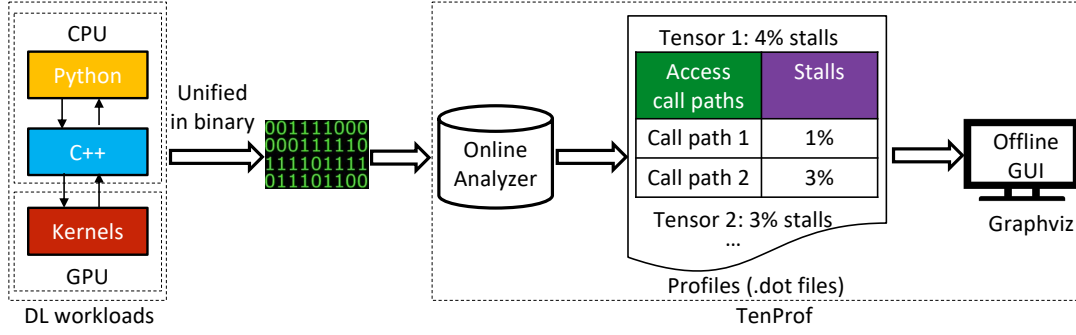


Figure 2: Overview of TENPROF.

4 Methodology and Implementation

This section drills down into TENPROF’s core component — online analyzer, which centers around four tasks. First, TENPROF represents tensor memory layouts, establishing the foundation for the subsequent analyses. Second, it traces and models tensor transformations by monitoring the input and output tensors of Aten operators. Third, TENPROF captures and characterizes tensor accesses by attributing memory instructions to their corresponding base or view tensors. Finally, it correlates memory stalls collected through PC sampling with these accesses, thereby identifying inefficient tensor accesses and tracing them back to the transformations that introduced them.

4.1 Representing Tensor Memory Layouts

Accurately representing a tensor’s memory layout is a prerequisite for tracing its transformations (Section 4.2) and capturing its accesses (Section 4.3). Computing the address range for a base tensor is simple and fast ($O(1)$ time) because it owns a single contiguous address range $[\text{start address}, \text{start address} + \text{size})$ to hold all of its elements. However, determining the memory layout of a view tensor is challenging. View tensors may exhibit non-contiguous layouts due to metadata-only transformations (e.g., slicing) that modify strides. As a result, the elements of a view tensor can be scattered across multiple disjoint address ranges.

Consider an n -dimensional view tensor T with the following metadata:

- `start_addr`: T ’s start address,
- `sizes[d]`: the number of elements along dimension d ,
- `strides[d]`: the step (in elements) to advance by one position along dimension d ,
- `elem_size`: the size (in bytes) of a single element, determined by the tensor’s data type (e.g., 4 bytes for float32).

Naive approach. A straightforward approach computes the address of every element in a view tensor using the tensor’s start address, strides, and element size, and then merges adjacent addresses into contiguous ranges. Specifically, the address of an element $T_{i_0, i_1, \dots, i_{n-1}}$ is computed as follows:

$$\text{start_addr} + \sum_{d=0}^{n-1} i_d \times \text{strides}[d] \times \text{elem_size} \quad (1)$$

where i_d denotes the index of the element along dimension d .

While correct, this approach is computationally expensive, requiring $\prod_{d=0}^{n-1} \text{sizes}[d]$ address computations. For large tensors, particularly those with millions of elements, this exhaustive enumeration becomes prohibitively slow.

Improved approach. We optimize by recognizing that contiguous inner dimensions can be collapsed into a single linear block, allowing iteration only over outer non-contiguous dimensions. Specifically, we identify the *innermost dimension whose stride deviates from contiguity*, i.e., the innermost dimension d_s such that

$$\text{strides}[d_s] \neq \text{sizes}[d_s + 1] \times \text{strides}[d_s + 1]. \quad (2)$$

We refer to d_s as the *split dimension*. For any fixed index of the split dimension and all outer dimensions (i.e., $d < d_s$), the elements along the inner dimensions (i.e., $d > d_s$) occupy a single contiguous block in memory. Therefore, instead of iterating over all elements in a view tensor, we enumerate only the combinations of indices in the outer non-contiguous dimensions, and for each combination, emit one contiguous memory block covering the inner contiguous block. The address computation becomes:

$$\text{inner_block_addr} + \sum_{d=0}^{d_s} i_d \times \text{strides}[d] \times \text{elem_size} \quad (3)$$

where `inner_block_addr` is pre-computed once from the collapsed inner contiguous block. This improved approach reduces the number of address computations from $\prod_{d=0}^{n-1} \text{sizes}[d]$ to $\prod_{d=0}^{d_s} \text{sizes}[d]$ as it iterates over only the $\prod_{d=0}^{d_s} \text{sizes}[d]$ combinations of outer indices.

For simplicity, consider a 3-dimensional view tensor with sizes (4, 5, 10) and strides (80, 12, 1) in elements. Dimension 1 is non-contiguous because its stride (=12) does not equal the product of the size (=10) and stride (=1) of the faster-changing dimension 2. The split dimension is therefore dimension 1. For each of the 4 indices in dimension 0, we enumerate the 5 indices in dimension 1, each producing a contiguous address range that covers 10 elements in dimension 2. This yields a total of 20 (=4×5) address computations, compared to 200 (=4×5×10) address computations required by the *naive approach*.

Extension for transposed tensors. The above optimization assumes that a view tensor’s logical dimension order matches its

physical stride order, i.e., outer dimensions have larger strides than inner dimensions in a row-major storage mode. However, a transposed tensor reorders dimensions of its base tensor, resulting in a mismatch between its dimension and stride order. Directly applying the aforementioned *improved approach* in such cases can therefore risk collapsing the wrong dimensions. To address this, we reorder a view tensor's logical dimensions by sorting them in descending order of stride magnitude. This reordering reconstructs the tensor's dimension order prior to transposition, thereby restoring the correspondence between dimension order and contiguity. The *improved approach* is then applied to the reordered dimensions, ensuring that only physically contiguous inner dimensions are collapsed, even when the view tensor is produced by arbitrary transpositions.

4.2 Tracing Tensor Transformations

In PyTorch, Aten operators are responsible for creating, transforming, and computing on tensors by consuming input tensors and producing output tensors. By tracing these operators, TENPROF models tensor transformations as a *forest*, where each tree consists of a base tensor (root) and view tensors (descendants) transformed from it. Each node holds the tensor's address range(s) and metadata. Edges encode transformation relationships among tensors and are of two types: intra-tree and cross-tree edges. An intra-tree edge represents a view transformation, linking a parent tensor to its derived view tensor. An inter-tree edge arises when an Aten operator produces a base tensor, thereby initiating a new transformation tree. We next describe the insertion and deletion procedures used to construct the transformation forest.

Insertion operations. Algorithm 1 outlines the forest insertion procedure. Using the PyTorch RecordFunctionCallback API [62], TENPROF intercepts both the call and return events of each Aten operator to capture its input and output tensors (lines 4 and 10 in Algorithm 1), respectively. For each input tensor, TENPROF directly looks up its address range(s) in the forest without recomputation (line 6), since it was previously inserted when the Aten operator that produced it returned. For each output tensor, TENPROF computes its address range(s) from metadata (line 12), as detailed in Section 4.1. The output tensor is either a base tensor or a view of one of the input tensors. This is determined by checking whether the output tensor's address range(s) lies within that of any input tensor (line 14). If no such enclosing input tensor exists, the output tensor is a base tensor, and TENPROF creates a transformation tree with it as the root (lines 15-19). Otherwise, TENPROF inserts the output tensor as a child node of the enclosing input tensor into the forest (lines 20-22), representing a view transformation.

Deletion operations. In PyTorch, GPU memory management is handled by the CUDA Caching Allocator in the C10 library [59], rather than directly by Aten operators. When the final reference to a base tensor is released, and no view tensors remain associated with it, the Caching Allocator reclaims the memory block containing the base tensor. This block may subsequently be reused to avoid the overhead of repeated `cudaMalloc/cudaFree` calls. To enable fine-grained tracking of memory usage, C10 exposes the `MemoryReportingInfoBase` API [58], which notifies profilers of allocation and deallocation events in the Caching Allocator. TENPROF

Algorithm 1 Transformation forest insertion in TENPROF

```

1: procedure INSERTTENSORTRANSFORMATION(op)  ▷ op: Aten operator
2:   /* Step 1: Intercept call event */
3:   on_call_event(op):
4:     inputs ← op.GETINPUTTENSORS()
5:     for tin ∈ inputs do
6:       tin.addr_ranges ← FORESTLOOKUPADDRESSRANGES(tin.metadata)
7:     end for
8:   /* Step 2: Intercept return event */
9:   on_return_event(op):
10:    outputs ← op.GETOUTPUTTENSORS()
11:    for tout ∈ outputs do
12:      tout.addr_ranges ← COMPUTEADDRESSRANGES(tout.metadata)  ▷ Sec. 4.1
13:    /* Step 3: Classification */
14:    enclosing_input ← FINDENCLOSINGINPUT(inputs, tout.addr_ranges)
15:    if enclosing_input is None then  ▷ Case: base tensor
16:      CREATETREE(root = tout)
17:      if inputs ≠ ∅ then
18:        MARKEDGE(tout, inputs, "cross-tree")
19:      end if
20:    else  ▷ Case: view tensor
21:      INSERTCHILD(parent = enclosing_input, child = tout)
22:      MARKEDGE(tout, enclosing_input, "intra-tree")
23:    end if
24:  end for
25: end procedure

```

registers a callback through this API to monitor deallocations. Upon invocation, the callback records the address range of the freed memory block. TENPROF then locates the base tensor whose address range falls within this block and removes its transformation tree from the forest, as illustrated in Figure 3. This ensures the forest accurately reflects the dynamic lifetime of tensors during execution.

4.3 Capturing Tensor Accesses

Tensors are ultimately accessed within GPU kernels invoked by Aten operators³. TENPROF instruments memory instructions inside these kernels. During execution, each instrumented instruction records the memory addresses it accesses. By comparing these addresses with the known address ranges of the Aten operator's input tensors⁴, TENPROF identifies which tensor and which range in the tensor have been accessed.

A straightforward design would log addresses accessed by memory instructions into a preallocated GPU buffer, transfer the buffer to the CPU once full, perform instruction-to-tensor matching there, and then reuse the buffer for logging subsequent accessed addresses. However, this approach introduces frequent GPU–CPU transfers

³An Aten operator may dispatch multiple GPU kernels.

⁴Each input tensor's address range(s) is maintained in the transformation forest, inserted when the Aten operator that produced it returned, as detailed in Section 4.2.

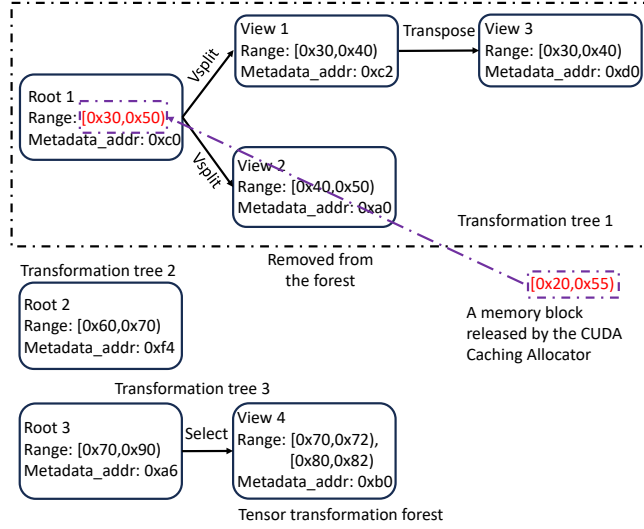


Figure 3: Transformation forest deletion in TENPROF. When the CUDA Caching Allocator frees a memory block, TENPROF identifies the base tensor whose storage resides in the freed block and removes the entire transformation tree rooted at that tensor.

and fails to exploit the GPU’s parallelism for efficient instruction-to-tensor matching.

To address this limitation, TENPROF performs instruction-to-tensor matching directly on the GPU, as shown in Figure 4. On the CPU, TENPROF builds an address map $\mathcal{A}$ for each Aten operator, a sorted array of address ranges corresponding to the operator’s input tensors. For each GPU kernel launched by the Aten operator, TENPROF allocates a binary hit matrix $\mathcal{H} \in \{0, 1\}^{N_{\text{inst}} \times N_{\text{range}}}$, where each row corresponds to an instrumented memory instruction in the kernel and each column corresponds to an address range in $\mathcal{A}$. An entry $\mathcal{H}_{i,j} = 1$ if the $(i+1)^{\text{th}}$ instrumented memory instruction hits the $(j+1)^{\text{th}}$ address range in $\mathcal{A}$, and $\mathcal{H}_{i,j} = 0$ otherwise. All the entries in $\mathcal{H}$ are initialized to zero (“no-hit”). At kernel launch, both $\mathcal{A}$ and $\mathcal{H}$ are transferred to the GPU. Each instrumented memory instruction performs a binary search over $\mathcal{A}$ to identify the address range containing the accessed addresses and updates the corresponding entry in $\mathcal{H}$ to 1 (“hit”). After kernel completion, $\mathcal{H}$ is copied back to the CPU, where TENPROF aggregates the results to determine which tensors and which ranges in these tensors were accessed by which instructions.

This GPU-side matching design significantly reduces profiling overhead. For example, it lowers TENPROF’s end-to-end profiling time for the demucs-eval benchmark in TorchBench [37] to 4 minutes, compared to 28 minutes with CPU-side matching.

4.4 Exploring Tensor Access Efficiency

In addition to tracing tensor accesses, TENPROF identifies tensors that are particularly *costly* to access and prioritizes them for user inspection. It does so by attributing memory instructions that incur hardware stalls to the tensors they access. Concretely, TENPROF

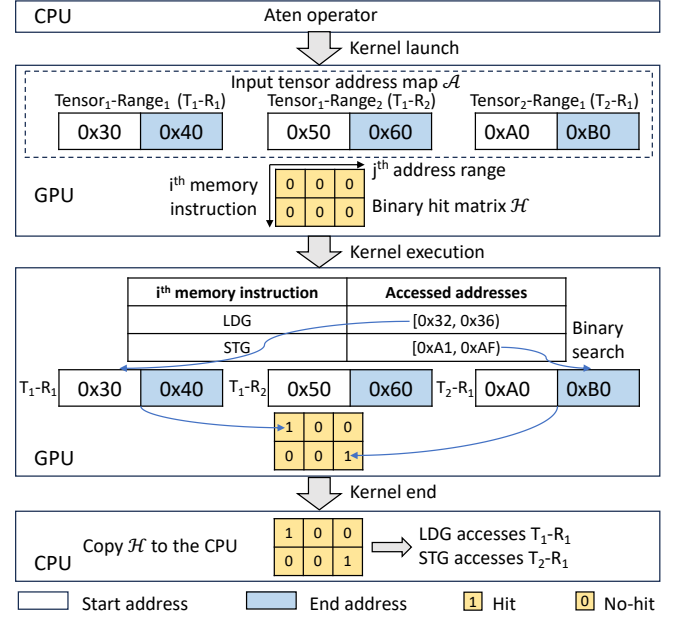


Figure 4: Instruction-to-tensor matching. Tensor_i-Range_j denotes the j^{th} address range of Tensor_i. Each instrumented memory instruction records its accessed addresses during kernel execution. These addresses are compared against the address ranges of input tensors, producing a binary hit matrix that identifies which tensor ranges are accessed. The mapping enables TENPROF to attribute memory instructions back to specific tensors and their transformations.

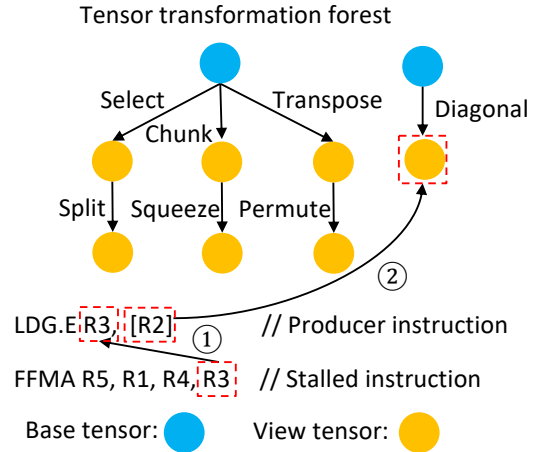


Figure 5: Stall-to-tensor matching. The FFMA instruction stalls on register R3, which is produced by the LDG.E instruction loading from memory address [R2]. ① TENPROF shifts stall attribution from the FFMA to the LDG.E to identify the true source of memory dependency stalls. ② TENPROF then attributes the LDG.E and its stall counts to the accessed tensor, quantifying the stall cost of accessing that tensor.

leverages CUPTI PC Sampling API [43] to collect memory stall-related instructions within GPU kernels launched by Aten operators, including stalls from memory throttling and memory dependencies [49].

Memory throttling stalls are directly attributed to the sampled (stalled) instructions. In contrast, memory dependency stalls occur when an instruction must wait for data produced by a preceding *producer* instruction. As illustrated in ① of Figure 5, the FFMA instruction stalls on register R3, whose value is produced by a preceding LDG.E instruction that loads from memory address [R2]. To identify the true source of dependency stalls, TENPROF applies a backward slicing algorithm [8] to extract the def-use chains of dependency-stalled instructions. This shifts stall attribution from the stalled instruction (the use site) to its producer instruction (the definition site). Accordingly, in Figure 5, TENPROF reassigns the stall attribution from the FFMA to the LDG.E.

Building on the instruction-to-tensor mapping established in Section 4.3, TENPROF aggregates inefficient memory instructions — *stalled instructions* for throttling stalls and *producer instructions* for dependency stalls — along with their associated stall counts, into the tensors they access, as shown in ② of Figure 5. The result is a per-tensor stall-weighted metric, termed the *Tensor Stall Cost (TSC)*, which quantifies each tensor’s contribution to overall memory stalls. Users can then explore tensors with high TSC values within the transformation forest to diagnose inefficiencies.

A practical challenge that TENPROF must address is the incompatibility between memory instrumentation and PC sampling within a single profiling pass. Instrumentation interposes every memory instruction by executing instrumented kernels, which incurs orders-of-magnitude slowdown (10–100×). This heavy perturbation fundamentally alters program execution, invalidating the statistical assumptions of PC sampling. Instead of capturing the genuine stall behavior of the original kernel, PC sampling in this setting would predominantly reflect the artificially inflated stalls introduced by the instrumentation code itself. To address this limitation, TENPROF provides two alternative profiling modes: *application replay* and *kernel replay*.

In application-replay mode, PC sampling and memory instrumentation are separated into two independent TENPROF clients. An application is first run with the PC-sampling client and then rerun with the memory-instrumentation client. TENPROF correlates results across the two runs to attribute stalled memory instructions to the tensors they access. In kernel-replay mode, TENPROF leverages the CUPTI Checkpoint API [42] to decouple the two mechanisms at the kernel level, which captures a kernel launch, together with its inputs, control parameters, and execution state, into a checkpoint. The captured kernel can then be replayed multiple times from a consistent state under different profiling configurations. Specifically, TENPROF performs two complementary profiling passes per kernel: one with PC sampling enabled to identify memory instructions that cause stalls and another with memory instrumentation enabled to attribute these instructions to the tensors they access.

Both replay modes align with standard practices in state-of-the-art GPU profilers such as NVIDIA’s Nsight Compute [45] and exhibit distinct trade-offs. Application replay better preserves the natural end-to-end execution context, as certain microarchitectural states (e.g., cache contents) are not saved in kernel checkpoints.

However, this fidelity comes at the cost of higher overhead, since the entire application, including both CPU and GPU execution, are re-executed. In contrast, kernel replay is more efficient, as it re-executes only GPU kernels, but may miss microarchitectural states. In practice, users can select the replay mode that better balances profiling fidelity and overhead for their specific use case.

5 Evaluation

We evaluate TENPROF on an NVIDIA RTX 4090 GPU, with the detailed configuration summarized in Table 1.

Time overhead. We evaluate TENPROF’s runtime overhead on 25 benchmarks from TorchBench [37], a state-of-the-art DL benchmark suite developed by the PyTorch team. The overhead of each benchmark is measured as the ratio of its execution time with TENPROF enabled to its native execution time. Figure 6 presents the results. Across all benchmarks, TENPROF incurs a geometric mean overhead of 26× and a median overhead of 29×, as shown by the blue bars. To understand the primary source of this overhead, we implemented a minimal baseline tool that enables only memory instrumentation and PC sampling, without performing any additional analysis or bookkeeping at instrumented/sampled instructions. As shown by the orange bars, this baseline exhibits a geometric mean overhead of 20× and a median overhead of 24×.

These results indicate that memory instrumentation and PC sampling account for the majority of the observed overhead, which represent the inherent cost of obtaining fine-grained performance insights at this level of fidelity. The additional overhead introduced by TENPROF’s tensor-centric analysis is comparatively modest.

Finally, we note that DL training workloads exhibit substantial iteration-level repetition. In practice, profiling only a small number of iterations is often sufficient to capture the dominant performance characteristics, thereby effectively amortizing the profiling overhead over long-running training jobs with millions of iterations.

6 Case Studies

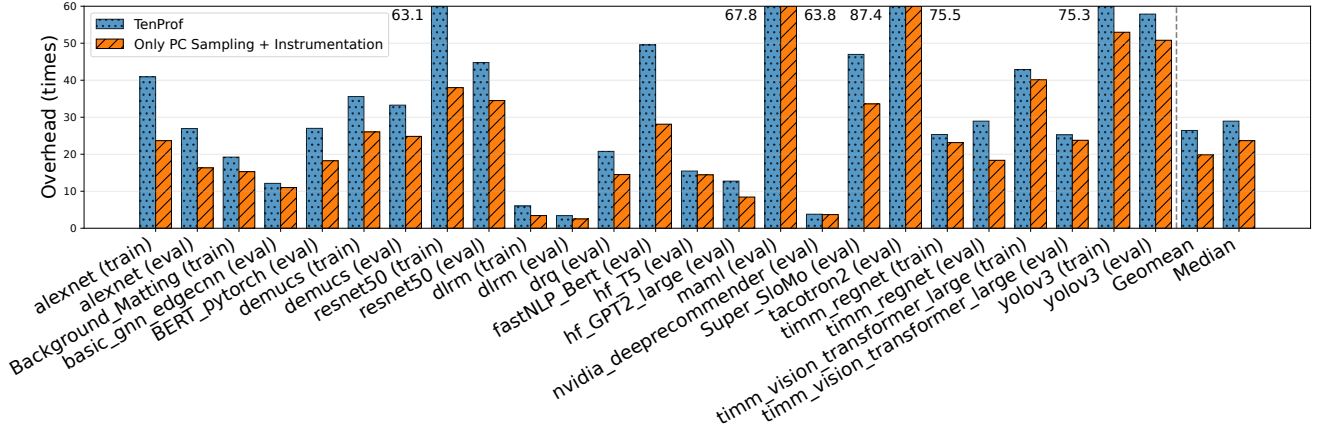
Table 2 summarizes the inefficiencies identified by TENPROF in five DL applications, including Docling [9], Hunyuan3D-2 [22], Sana [48], OpenOCR [10], and TritonBench [31], along with the speedups achieved by eliminating them. Sana was already introduced in Section 1; in the following, we describe the remaining four applications in detail.

6.1 Docling

Docling [9] is a generative AI ecosystem that provides document processing, parsing, and understanding. Figure 7 shows a snapshot of TENPROF’s GUI, which visualizes a portion of the tensor transformation forest produced by TENPROF’s online analyzer when profiling an execution of Docling. The visualization distinguishes two types of nodes: ellipses with solid outlines denote base tensors, while ellipses with dashed outlines denote view tensors. Directed edges encode transformation relationships among tensors. We further differentiate two types of edges: solid edges represent transformations that involve tensor accesses, while dashed edges represent transformations that do not. Each node and edge is annotated with a unique numeric identifier, and hovering over an element reveals

Table 1: Experimental platform configurations.

CPU	DRAM	GPU	GPU memory	System	GPU driver	CUDA Toolkit	Python	PyTorch	GCC
Intel(R) Xeon(R) Silver 4410Y	512 GB DDR5	RTX 4090	24 GB GDDR6X	Linux 6.8	570.86.10	12.8.61	3.11.9	2.6.0-cu126	10.5.0

**Figure 6: TENPROF's time overhead.****Table 2: Overview of performance optimizations guided by TENPROF. ITTT stands for intra-tree tensor transformation. CTTT stands for cross-tree tensor transformation. TSC stands for tensor stall cost, a stall-weighted metric that quantifies each tensor's contribution to overall memory stalls.**

DL Applications	Modules	Problematic tensors			Optimizations	Module-level speedups	Whole-application speedups
		Names	TSC	Patterns			
Docling	TMTransformerDecoderLayer	kv_proj	2%	CTTT	Remove unnecessary transformations	2.71×	1.16×
Hunyuan-3D2	GELU	x	1%	CTTT	Remove the contiguous operator	1.55×	1.02×
Sana	SanaLinearAttnProcessor2_0	hidden_states	1%	ITTT CTTT	Materialize the tensor into a contiguous buffer	1.06×	1.04×
OpenOCR	EncoderWithSVTR	z	2%	ITTT	Materialize the tensor into a contiguous buffer	1.07×	1.04×
TritonBench	torch_compile_embedding	in_ptr1	31%	ITTT	Tile-based memory accesses	1.17×	1.13×

a tooltip that displays its corresponding call path along with its percentage contribution to whole-application memory stalls.

To help users quickly identify stall-intensive tensors, we encode quantitative information into the visualization. Specifically, the size of each node is proportional to the associated tensor's percentage contribution to whole-application memory stalls, while the thickness of each edge is proportional to the associated transformation's percentage contribution. This visual encoding enables users to immediately spot the most problematic tensors and transformations, as larger nodes and thicker edges naturally attract visual attention. The design is especially effective when the GUI contains many tensors and transformations, as it reduces cognitive load.

Listing 2 shows the code snippet corresponding to Figure 7. Base tensor 617 (kv_proj) is created at line 3, followed by a series of transformations at line 4: 443 (unflatten()), 474 (unsqueeze()), 481 (transpose()), 496 (squeeze()), and finally 500 (contiguous()). The tooltip in Figure 7 highlights that tensor accesses associated

with transformation 500 (contiguous()) account for 2% of total memory stalls. This operation materializes view tensor 627 into a new base tensor 641. However, neither base tensor 641 nor its derived view tensors 645 (kv_proj[0]) and 647 (kv_proj[1]) are subsequently accessed, as evidenced by the absence of solid outgoing edges from these tensors in the visualization. This observation reveals that transformation 500 is redundant.

Furthermore, tracing the transformation lineage of view tensor 627 shows that none of its ancestor tensors are accessed. This implies that the entire transformation chain introduced at lines 3-5 of Listing 2 is unnecessary. Eliminating this redundant chain removes superfluous computation and memory traffic, yielding a 2.71× speedup in the TMTransformerDecoderLayer module.

```

1 def _in_projection_packed(q: Tensor, ...) -> list[
  Tensor]:
2     q_proj = linear(q, w_q, b_q)
3     kv_proj = linear(k, w_kv, b_kv)

```

```

4 - kv_proj = kv_proj.unflatten(-1, (2, E)).unsqueeze
    (0).transpose(0, -2).squeeze(-2).contiguous()
5 - return (q_proj, kv_proj[0], kv_proj[1])
6 + return (q_proj, None, None)

```

Listing 2: Optimization to the `_in_projection_packed` function in Docling. The original code applies a sequence of view transformations followed by `contiguous()`, creating a tensor that is never accessed. Removing these unnecessary transformations eliminates computation waste and saves memory traffic, yielding a $2.71\times$ module-level speedup.

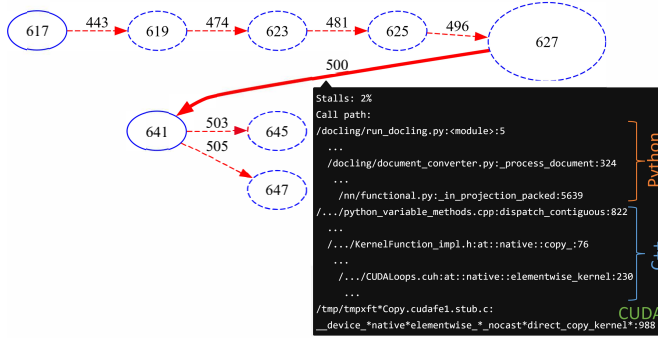


Figure 7: A snapshot of TENPROF’s GUI that visualizes a portion of the profile (tensor transformation forest) produced by TENPROF’s online analyzer when profiling Docling. The visualization highlights a transformation from view tensor 627 to base tensor 641 that incurs 2% of total memory stalls.

6.2 Hunyuan3D-2

Hunyuan3D-2 [22] is a suite of large-scale diffusion models for high-resolution 3D asset generation. Profiling with TENPROF reveals that 1% of total memory stalls are attributed to a `contiguous()` operator applied at line 6 of Listing 3 to the view tensor `x`, which has a non-contiguous memory layout. This operator materializes `x` into a new base tensor with contiguous storage, incurring a full tensor copy.

Inspection of the transformation forest reveals that the materialized tensor is consumed only once by the immediately following `gelu()` invocation at line 6, indicating that the cost of materialization likely outweighs any benefit from improved memory locality. This inefficiency is subtle: without tensor-level stall attribution, developers may reasonably assume that `contiguous()` consistently improves performance, whereas in this context it introduces unnecessary overhead. Guided by TENPROF, removing `contiguous()` (line 7) eliminates the unnecessary tensor copy and yields a $1.55\times$ speedup in the GELU module. This optimization has been confirmed and upstreamed by the Hunyuan3D-2 developers [21].

```

1 class GELU(nn.Module):
2     def __init__(self, approximate='tanh'):
3         super().__init__()
4         self.approximate = approximate
5     def forward(self, x: Tensor) -> Tensor:
6 -         return nn.functional.gelu(x.contiguous()),
          approximate=self.approximate)

```

```

7 +         return nn.functional.gelu(x, approximate=self.
          approximate)

```

Listing 3: Optimization to the GELU module in Hunyuan3D-2. The original code eagerly materializes the view tensor `x` with `contiguous()`, incurring a full copy despite being used only once. Removing this operation avoids the unnecessary materialization and yields a $1.55\times$ speedup.

6.3 OpenOCR

OpenOCR [10] is a general optical character recognition (OCR) system that targets both high accuracy and efficiency. Profiling with TENPROF attributes 2% of total memory stalls to the tensor `z` at line 8 of Listing 4, ranking it among the top 5% most stall-intensive tensors. The profile also shows that `z` is repeatedly accessed within the `svtr_block` loop (line 7), indicating significant reuse.

The inefficiency arises because `z` is a non-contiguous view tensor. Such layouts lead to strided, non-coalesced memory accesses, which substantially degrade GPU efficiency. To remedy this, we insert a `contiguous()` call at line 6 to materialize `z` into a new base tensor with contiguous storage. While this operation introduces a one-time copy cost, it ensures that all subsequent accesses to `z` are contiguous and coalesced. Because `z` is reused across multiple loop iterations, the savings from faster memory accesses outweigh the copy overhead. With this modification, the `EncoderWithSVTR` module achieves a $1.07\times$ speedup. This optimization has been validated and merged by the OpenOCR developers [52].

This case highlights a broader insight: `contiguous()` can either harm or help performance depending on reuse. When the materialized tensor is consumed only once or a handful of times as in Hunyuan3D-2 (Section 6.2), the copy overhead dominates and should be avoided. In contrast, when the tensor is reused multiple times as in OpenOCR, the improved memory coalescing can amortize the copy cost and deliver measurable speedups.

```

1 class EncoderWithSVTR(nn.Module):
2     ...
3     def forward(self, x):
4         ...
5 -         z = z.flatten(2).transpose(1, 2)
6 +         z = z.flatten(2).transpose(1, 2).contiguous()
7         for blk in self.svtr_block:
8             z = blk(z)
9         ...
10        return z

```

Listing 4: Optimization to the `EncoderWithSVTR` module in OpenOCR. In the original code, tensor `z` is a non-contiguous view after a series of transformations at line 5, causing strided and inefficient memory accesses in the subsequent `svtr_block` loop. By adding `contiguous()`, `z` is materialized into a contiguous base tensor, enabling coalesced memory accesses across loop iterations.

```

1 @triton.jit
2 def triton_poi_fused_embedding_0(
3     in_ptr0, in_ptr1, out_ptr0, XBLOCK: tl.constexpr):
4 +     in_ptr0, in_ptr1, out_ptr0, YBLOCK: tl.constexpr,
        XBLOCK: tl.constexpr):
5     xoffset = tl.program_id(0) * XBLOCK
6     xindex = xoffset + tl.arange(0, XBLOCK)[: ]
7 +     yoffset = tl.program_id(0) * YBLOCK
8 +     yindex = yoffset + tl.arange(0, YBLOCK)[None, :]
9 +     xoffset = tl.program_id(1) * XBLOCK

```

```

10 +   xindex = xoffset + tl.arange(0, XBLOCK)[: , None]
11 -   x1 = xindex // 4096
12 -   x0 = xindex % 4096
13 -   x2 = xindex
14 +   y0 = yindex
15 +   x1 = xindex
16 -   tmp0 = tl.load(in_ptr0 + (x1), None, ...)
17 +   tmp0 = tl.load(in_ptr0 + (y0), None, ...)
18 -   tmp1 = tl.full([XBLOCK], 8192, tl.int32)
19 +   tmp1 = tl.full([XBLOCK, YBLOCK], 8192, tl.int32)
20   tmp2 = tmp0 + tmp1
21   tmp3 = tmp0 < 0
22   tmp4 = tl.where(tmp3, tmp2, tmp0)
23 -   tmp6 = tl.load(in_ptr1 + (x0 + 4096 * tmp4), None)
24 +   tmp6 = tl.load(in_ptr1 + (x1 + 4096 * tmp4), None)
25   ...

```

Listing 5: Specialized Triton kernel generated for the `torch_compile_embedding` operator. In the baseline 1-D grid implementation, program instances access different rows of the embedding table `in_ptr1`, resulting in poor L2 cache reuse and frequent memory stalls. By restructuring the kernel into a 2-D grid, the optimized implementation enables concurrent program instances to share rows of `in_ptr1` in the L2 cache, reducing memory traffic and yielding a 1.17× speedup.

6.4 PyTorch Embedding Operator

Since PyTorch 2.0, eager-mode operators can be compiled into highly optimized Triton kernels, enabling efficient execution on modern GPUs [4]. Among these operators, the embedding operator is particularly critical in large language models. It maps discrete token indices into a dense vector space and is heavily used to retrieve tensor representations during model training and inference.

We profile the `torch_compile_embedding` operator in inductor mode using TritonBench [31]. Listing 5 shows the Triton kernel generated for this operator. The kernel takes as input three tensors — `in_ptr0` (index tensor), `in_ptr1` (embedding table), and `out_ptr0` (output tensor) — and a compile-time parameter `XBLOCK` to perform an embedding lookup. Using TENPROF, we observe that 31% of total memory stalls are attributed to loads from the embedding table `in_ptr1`, concentrated at line 23.

In the baseline implementation, the kernel is launched using a 1-D grid parameterized by `XBLOCK`. The embedding table `in_ptr1`, which is logically 2-D, is flattened into a 1-D linear index space (line 6). However, this 1-D linearized iteration order leads to poor temporal locality when accessing `in_ptr1` at line 23: program instances⁵ that are launched concurrently on different SMs access entirely disjoint sets of rows in `in_ptr1`. Consequently, each row is typically brought into the L2 cache only once before being evicted, and subsequent accesses to the same row by other program instances trigger additional global memory fetches. This access pattern leads to elevated L2 cache miss rates, which manifest as frequent memory stalls.

The optimized kernel restructures the computation into a 2-D launch configuration parameterized by `XBLOCK` and `YBLOCK`. Each program instance now computes a rectangular tile of shape `[XBLOCK, YBLOCK]`, where `yindex` enumerates embedding lookups (line 8) and `xindex` enumerates contiguous columns within each embedding vector (line 10). Crucially, this 2-D tiling ensures that all

program instances along the X dimension (processing different column slices) access the same set of rows in `in_ptr1` at line 24. When these program instances execute concurrently, the first access to a row brings it into the L2 cache, allowing subsequent instances to reuse the cached data instead of issuing additional global memory accesses. The increased cache reuse reduces memory stalls and yields a measured 1.17× speedup over the baseline kernel.

7 Discussion

As with most dynamic profilers, the results produced by TENPROF depend on the specific program inputs. Accordingly, using inputs that reflect typical execution scenarios is essential for obtaining representative and actionable insights. In addition, while TENPROF is effective at uncovering performance inefficiencies, it does not automatically apply code transformations. Instead, users are responsible for determining suitable and safe optimization strategies. Finally, TENPROF focuses on inefficiencies related to tensor accesses and transformations. Consequently, it does not capture inefficiencies orthogonal to tensor behavior, such as those arising from excessive kernel launches (e.g., launching many small kernels instead of fewer fused kernels).

8 Related Work

Over the past decade, a rich ecosystem of profilers has emerged to measure, analyze, and optimize GPU performance, spanning vendor-provided profilers [2, 3, 45, 46], HPC profilers with GPU support [1, 29, 38], research-driven profilers [17, 20, 32, 71, 72], and DL profilers [14–16, 30, 35, 36, 44, 64]. Each class embodies distinct trade-offs among overhead, granularity, usability, and extensibility, reflecting the diverse requirements of GPU programmers, systems researchers, and DL practitioners.

Vendor-provided profilers. Vendor-provided toolchains form the de facto baseline for GPU performance analysis. NVIDIA’s Nsight Systems [46] offers system-level tracing to reveal interactions across CPU and GPU activities, while Nsight Compute [45] provides kernel-level metrics such as occupancy, instruction throughput, and memory bandwidth utilization. Complementing these tools, CUPTI [40] exposes a programmatic interface for collecting hardware performance counters, activity traces, and instruction stalls, forming the foundation of many research prototypes, including ours. Similarly, AMD offers ROCProfiler [2] and ROC-Tracer [3], which provide analogous profiling capabilities within the ROCm ecosystem.

HPC profilers with GPU support. State-of-the-art HPC profilers, such as HPCToolkit [1], TAU [38], and Score-P [29], extend their traditional CPU-side profiling to incorporate GPU kernel execution. These tools emphasize statistically robust, low-overhead techniques (e.g., asynchronous sampling) and can attribute GPU execution costs to call paths in large-scale applications, within and across compute nodes. Compared to vendor-provided profilers, HPC profilers are often better suited for long-running production workloads and multi-node analyses. However, they typically expose fewer GPU-specific microarchitectural diagnostics out of the box and depend on vendor APIs (e.g., CUPTI) for device-side visibility.

⁵A program instance is one execution of a Triton kernel, analogous to a CUDA thread block.

Research-driven profilers. Several academic efforts extend profiling to capture richer insights. GPA [72], a performance advisor, detects and characterizes GPU instruction stalls through CUPTI PC Sampling API and suggests potential code optimizations. DrGPU [17] extends GPA by collecting additional hardware performance metrics, such as cache misses, to better understand inefficient software-hardware interactions. GVProf [70], a value-aware profiler, detects redundant memory accesses within individual GPU kernels using Compute Sanitizer API. Their follow-up efforts include ValueExpert [71] and DrGPUM [32], which further refine this line of work. ValueExpert systematically characterizes patterns of memory-access redundancy, while DrGPUM focuses on identifying and classifying memory wastage patterns. More recently, Neutrino [20] proposes a GPU kernel profiling infrastructure that provides assembly-level probes to achieve high-fidelity, instruction-level analysis.

Despite their effectiveness, the aforementioned profilers are primarily designed for general-purpose GPU workloads (e.g., CUDA programs). As a result, they lack domain-specific abstractions needed to reason about DL workloads, such as tensors and the operators that create, transform, and consume them. Consequently, they cannot directly connect low-level hardware behavior to high-level DL semantics.

DL profilers. Built-in profilers shipped with DL frameworks (e.g., Kineto [36] and Traceme [16]), as well as framework-specific ones (e.g., PyTorch Profiler [35], JAX Profiler [15], and TensorBoard [14]), provide rich domain-specific information. In contrast, DLProf [44] is a portable DL profiler that supports both PyTorch and TensorFlow on NVIDIA GPUs, employing NVTX [47] annotations to mark interesting functions and their invocation relationships. Other tools, such as XSP [30] and Hotline Profiler [64], follow a similar approach, requiring user-based heavy instrumentation.

Orthogonal to these profilers, which adopt a code-centric perspective centered on DL operators, TENPROF takes a data(tensor)-centric approach by elevating tensors to first-class profiling entities. It systematically links tensor transformations, tensor-level memory accesses, and hardware performance metrics.

Data-centric profilers. Data-centric profiling [33], which focuses on performance impacts of data layouts [68], accesses [66], and placement [34], has a long history in computational research. Early applications of this approach were primarily adopted in scientific computing workloads, such as AMG [63] and LULESH [27], where it was used to dissect the intricate patterns of data allocation, dependency, and access across multiple CPU cores. Recently, there has been progress in applying data-centric profiling to GPU applications, including efforts to trace GPU data flows [73], monitor multi-GPU data transfers [24], and attribute performance metrics to GPU data objects [69]. These works target general-purpose GPU programs and do not fully explore the tight coupling among tensors, DL operators, and performance metrics.

9 Conclusions

This paper introduces TENPROF, the first tensor-centric profiler for guiding performance optimization in PyTorch-based DL workloads. In contrast to existing profilers, TENPROF offers an end-to-end view

of performance that directly connects low-level hardware stalls with the high-level DL semantics responsible for these inefficiencies. It achieves this by unifying three traditionally disjoint aspects of DL execution: tensor transformations, tensor-level memory accesses, and underlying hardware performance behavior. The resulting profiles are actionable, enabling developers to trace performance bottlenecks directly to specific tensor transformations. Through case studies on real-world DL applications, we show that resolving inefficiencies identified by TENPROF often requires only minor code changes yet delivers non-trivial performance improvements. Several of these optimizations have been validated and upstreamed by application developers, underscoring TENPROF's practical utility in production settings.

Acknowledgments

We thank all the anonymous reviewers for their valuable comments. This work was supported by the National Science Foundation under grants CNS-2125732, CNS-2441754, and DUE-2417470.

References

- [1] Laksono Adhianto, Sinchan Banerjee, Mike Fagan, Mark Krentel, Gabriel Marin, John Mellor-Crummey, and Nathan R Tallent. 2010. HPCToolkit: Tools for performance analysis of optimized parallel programs. *Concurrency and Computation: Practice and Experience* 22, 6 (2010), 685–701.
- [2] AMD. 2025. ROCProfiler. <https://rocm.docs.amd.com/projects/rocprowler>.
- [3] AMD. 2025. ROCTracer. <https://rocm.docs.amd.com/projects/roctracer>.
- [4] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsch, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 929–947. <https://doi.org/10.1145/3620665.3640366>
- [5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 578–594.
- [6] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR abs/1410.0759* (2014). arXiv:1410.0759 <http://arxiv.org/abs/1410.0759>
- [7] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ip-polito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. PaLM: Scaling Language Modeling with Pathways. *J. Mach. Learn. Res.* 24 (2022), 240:1–240:113. <https://api.semanticscholar.org/CorpusID:247951931>
- [8] Cristina Cifuentes and Antoine Fraboulet. 1997. Intraprocedural Static Slicing of Binary Executables. In *Proceedings of the International Conference on Software Maintenance (ICSM '97)*. IEEE Computer Society, USA, 188.
- [9] Deep Search Team. 2024. *Docling Technical Report*. Technical Report. arXiv:2408.09869 <https://arxiv.org/abs/2408.09869>

- [10] Yongkun Du, Zhineng Chen, Hongtao Xie, Caiyan Jia, and Yu-Gang Jiang. 2025. SVTRv2: CTC Beats Encoder-Decoder Models in Scene Text Recognition. In *ICCV*.
- [11] Edward Z. Yang. 2019. PyTorch internals. <https://blog.ezyang.com/2019/05/pytorch-internals>.
- [12] John Ellison and Emden Gansner. 2025. Graphviz. <https://graphviz.org>.
- [13] Free Software Foundation. 2025. The libunwind project. <https://www.nongnu.org/libunwind>.
- [14] Google. 2022. TensorFlow Profiler. https://www.tensorflow.org/tensorboard/tensorboard_profiling_keras.
- [15] Google. 2023. JAX Profiler. <https://jax.readthedocs.io/en/latest/jax.profiler.html>.
- [16] Google. 2025. Tensorflow trace. https://www.tensorflow.org/api_docs/python/tf/profiler/experimental/Trace.
- [17] Yueming Hao, Nikhil Jain, Rob Van der Wijngaart, Nirmal Saxena, Yuanbo Fan, and Xu Liu. 2023. DrGPU: A Top-Down Profiler for GPU Applications. In *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*. 43–53.
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [19] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. 2019. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF international conference on computer vision*. 1314–1324.
- [20] Songlin Huang and Chenshu Wu. 2025. Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association. <https://www.usenix.org/conference/osdi25/presentation/huang-songlin>
- [21] Hunyuan3D Team. 2025. Code patch in Hunyuan3D-2. <https://github.com/Tencent-Hunyuan/Hunyuan3D-2/issues/236>.
- [22] Hunyuan3D Team. 2025. Hunyuan3D 2.0: Scaling Diffusion Models for High Resolution Textured 3D Assets Generation. [arXiv:2501.12202 \[cs.CV\]](https://arxiv.org/abs/2501.12202)
- [23] IREE Organization. 2025. IREE. <https://iree.dev>.
- [24] Mohammad Kefah Taha Issa, Muhammad Aditya Sasongko, Ilyas Turimbetov, Javid Baydamirli, Dogan Sagbili, and Didem Unat. 2024. SnooPie: A Multi-GPU Communication Profiler and Visualizer. In *Proceedings of the 38th ACM International Conference on Supercomputing (Kyoto, Japan) (ICS '24)*. Association for Computing Machinery, New York, NY, USA, 525–536. <https://doi.org/10.1145/3650200.3656597>
- [25] Jason Ansel. 2022. TorchInductor. <https://dev-discuss.pytorch.org/t/torchinductor-a-pytorch-native-compiler-with-define-by-run-ir-and-symbolic-shapes/747>.
- [26] John M. Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Zidek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A A Kohl, Andy Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David A. Reiman, Ellen Clancy, Michal Zilinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. 2021. Highly accurate protein structure prediction with AlphaFold. *Nature* 596 (2021), 583 – 589.
- [27] Ian Karlin, Jeff Keasler, and J Robert Neely. 2013. *Lulesh 2.0 updates and changes*. Technical Report. Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States).
- [28] Jehad Khan, Paul Fultz, Artem Tamazov, Daniel Lowell, Chao Liu, Michael Melesse, Murali Nandhimandalam, Kamil Nasyrov, Ilya Perminov, Tejash Shah, Vasilii Filippov, Jing Zhang, Jing Zhou, Bragadeesh Natarajan, and Mayank Daga. 2019. MIOpen: An Open Source Library For Deep Learning Primitives. <https://arxiv.org/abs/1910.00078>
- [29] Andreas Knüpfer, Christian Rössel, Dieter Mey, Scott Biersdorff, Kai Diethelm, Dominic Eschweiler, Markus Geimer, Michael Gerndt, Daniel Lorenz, Allen Malony, Wolfgang Nagel, Yuri Oleyunik, Peter Philippen, Pavel Saviankou, Dirk Schmidt, Sameer Shende, Ronny Tschüter, Michael Wagner, Bert Wesarg, and Felix Wolf. 2012. Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir. In *Competence in High Performance Computing 2011*. Springer Berlin Heidelberg, 79–91.
- [30] Cheng Li, Abdul Dakkak, Jinjun Xiong, Wei Wei, Lingjie Xu, and Wen-mei Hwu. 2020. XSP: Across-stack profiling and analysis of machine learning models on GPUs. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 326–327.
- [31] Jianling Li, ShangZhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, WangHaojie WangHaojie, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 23053–23066. <https://aclanthology.org/2025.findings-acl.1183>
- [32] Mao Lin, Keren Zhou, and Pengfei Su. 2023. DrGPUM: Guiding Memory Optimization for GPU-Accelerated Applications. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 164–178.
- [33] Xu Liu and John Mellor-Crummey. 2013. A data-centric profiler for parallel programs. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–12.
- [34] Xu Liu and John Mellor-Crummey. 2013. Pinpointing data locality bottlenecks with low overhead. In *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 183–193.
- [35] Meta. 2023. PyTorch Profiler. https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html.
- [36] Meta. 2025. Kineto. <https://github.com/pytorch/kineto>.
- [37] Meta. 2025. TorchBench: A collection of open source benchmarks used to evaluate PyTorch performance. <https://github.com/pytorch/benchmark>.
- [38] Alan Morris, Allen D. Malony, Sameer Shende, and Kevin Huck. 2010. Design and Implementation of a Hybrid Parallel Performance Measurement System. In *Proceedings of the 2010 39th International Conference on Parallel Processing (ICPP '10)*. IEEE Computer Society, Washington, DC, USA, 492–501.
- [39] NVIDIA. 2022. Sanitizer API. <https://docs.nvidia.com/compute-sanitizer/SanitizerApi/table-of-contents.html>.
- [40] NVIDIA. 2025. CUDA Profiling Tools Interface (CUPTI). <https://docs.nvidia.com/cupti>.
- [41] NVIDIA. 2025. CUPTI Activity API. https://docs.nvidia.com/cupti/api/group__C_UPTI_ACTIVITY_API.html.
- [42] NVIDIA. 2025. CUPTI Checkpoint API. https://docs.nvidia.com/cupti/api/group__C_UPTI_CHECKPOINT_API.html.
- [43] NVIDIA. 2025. CUPTI PC Sampling API. https://docs.nvidia.com/cupti/api/group__C_UPTI_PCSAMPLING_API.html.
- [44] NVIDIA. 2025. DLProf User Guide. <https://docs.nvidia.com/deeplearning/frameworks/dlprof-user-guide>.
- [45] NVIDIA. 2025. NVIDIA Nsight Compute. <https://developer.nvidia.com/nsight-compute>.
- [46] NVIDIA. 2025. NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>.
- [47] NVIDIA. 2025. NVIDIA Tools Extension SDK. <https://github.com/NVIDIA/NVTX>.
- [48] NVIDIA. 2025. SANA: Efficient High-Resolution Image Synthesis with Linear Diffusion Transformer. <https://github.com/NVlabs/Sana>.
- [49] NVIDIA. 2025. Stall reasons mapping table. <https://docs.nvidia.com/cupti/main/main.html#stall-reasons-mapping-table>.
- [50] NVIDIA. 2026. Accelerating the future of autonomous vehicles. <https://www.nvidia.com/en-us/solutions/autonomous-vehicles>.
- [51] OpenAI. 2023. GPT-4 Technical Report. [arXiv:2303.08774 \[cs.CL\]](https://arxiv.org/abs/2303.08774)
- [52] OpenAI developers. 2025. Code patch in OpenOCR. <https://github.com/topdu/OpenOCR/pull/125>.
- [53] OpenXLA. 2025. OpenXLA. <https://openxla.org>.
- [54] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:202786778>
- [55] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of the Sixth Conference on Machine Learning and Systems*. mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html
- [56] Python Software Foundation. 2025. PyEval_GetFrame. https://docs.python.org/3/c-api/reflection.html#c.PyEval_GetFrame.
- [57] PyTorch Team. 2022. TorchDynamo. <https://github.com/pytorch/torchdynamo>.
- [58] PyTorch Team. 2025. C10_API MemoryReportingInfoBase. <https://github.com/pytorch/pytorch/blob/6e8865fbc161270e2f6c52817e6c667df417a3f7/c10/core/Allocator.h#L293>.
- [59] PyTorch Team. 2025. CUDA Caching Allocator. <https://github.com/pytorch/pytorch/blob/main/c10/cuda/CUDACachingAllocator.cpp>.
- [60] PyTorch Team. 2025. PyTorch codebase. <https://github.com/pytorch/pytorch>.
- [61] PyTorch Team. 2025. Tensor views. https://pytorch.org/docs/stable/tensor_view.html.
- [62] PyTorch Team. 2025. TORCH_API RecordFunctionCallback. https://github.com/pytorch/pytorch/blob/aeb5321b6360c899808d3461789b3bbd6265756e/aten/src/ATen/record_function.h#L129.
- [63] John W Ruge and Klaus Stüben. 1987. Algebraic multigrid. In *Multigrid methods*. SIAM, 73–130.
- [64] Daniel Snider, Fanny Chevalier, and Gennady Pekhimenko. 2023. Hotline Profiler: Automatic Annotation and A Multi-Scale Timeline for Visualizing Time-Use in DNN Training. *Proceedings of Machine Learning and Systems 5* (2023).

- [65] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19.
- [66] Shasha Wen, Lucy Cherkasova, Felix Xiazhu Lin, and Xu Liu. 2018. ProfDP: A lightweight profiler to guide data placement in heterogeneous memory systems. In *Proceedings of the 2018 International Conference on Supercomputing*. 263–273.
- [67] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. 2016. Aggregated Residual Transformations for Deep Neural Networks. <https://arxiv.org/abs/1611.05431>
- [68] Chao Yu, Probir Roy, Yuebin Bai, Hailong Yang, and Xu Liu. 2018. LWPTool: A lightweight profiler to guide data layout optimization. *IEEE Transactions on Parallel and Distributed Systems* 29, 11 (2018), 2489–2502.
- [69] Hui Zhang and Jeffrey Hollingsworth. 2019. Understanding the performance of GPGPU applications from a data-centric view. In *2019 IEEE/ACM International Workshop on Programming and Performance Visualization Tools (ProTools)*. IEEE, 1–8.
- [70] Keren Zhou, Yueming Hao, John Mellor-Crummey, Xiaozhu Meng, and Xu Liu. 2020. GVProf: A value profiler for GPU-based clusters. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [71] Keren Zhou, Yueming Hao, John Mellor-Crummey, Xiaozhu Meng, and Xu Liu. 2022. ValueExpert: Exploring Value Patterns in GPU-Accelerated Applications. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 171–185. <https://doi.org/10.1145/3503222.3507708>
- [72] Keren Zhou, Xiaozhu Meng, Ryuichi Sai, and John Mellor-Crummey. 2021. GPA: A GPU performance advisor based on instruction sampling. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 115–125.
- [73] Pierre Zins and Michel Dagenais. 2019. Tracing and profiling machine learning dataflow applications on GPU. *International Journal of Parallel Programming* 47, 5-6 (2019), 973–1013.